Machine Learning: Unit 1 (Detailed Notes)

Foundations of Machine Learning

Contents: 1. Intro & Workflow · 2. Types of ML · 3. Understanding Data · 4. Pandas · 5. Data Cleaning · 6. EDA · 7. Visualization · 8. Practice Questions

---

1. Introduction to Machine Learning

1.1 What is ML?

Machine Learning is a field of Artificial Intelligence in which algorithms learn patterns from data and use them to make predictions or decisions on new data, without being explicitly programmed with rules.

Mitchell's formal definition (1997): A program learns from experience **E** with respect to task **T** and performance measure **P**, if its performance at T, as measured by P, improves with E. *Example (spam filter):* T = classify emails; E = emails labeled spam/not spam; P = % classified correctly.

1.2 AI vs ML vs Deep Learning vs Data Science

Term	Meaning
AI	Machines that mimic human intelligence (broadest field)
ML	Subset of AI: learns from data
Deep Learning	Subset of ML: multi-layer neural networks
Data Science	Extracting insights from data using statistics, ML, visualization

1.3 Traditional Programming vs ML

- **Traditional:** Data + **Rules** → Output (programmer writes rules)
- **ML:** Data + **Output** → Rules/Model (algorithm discovers rules)

1.4 Why ML? Applications

- Rules too complex to write (face recognition)
- Rules change constantly (fraud detection)
- Personalization at scale (recommendations)
- Applications: healthcare diagnosis, banking (credit scoring), e-commerce, self-driving cars, NLP (chatbots), agriculture (crop yield).

1.5 Basic Terminology

Term	Meaning	Example
Dataset	Collection of data	Student records table
Instance / Sample / Row	One observation	One student

Feature / Attribute / Column / Input (X)	Measurable property	Age, marks
Target / Label / Output (y)	Value to predict	Pass/Fail
Model	Learned function f: X → y	Trained regression
Training	Learning from data	`model.fit()`
Testing	Evaluating on unseen data	`model.predict()`
Hyperparameter	Set before training	k in k-NN
Parameter	Learned from data	Slope in regression

1.6 The ML Workflow (Step-by-Step)

```
Problem Definition → Data Collection → Data Cleaning → EDA → Feature Engineering
      → Model Selection → Training → Evaluation → Tuning → Deployment → Monitoring
```

1. **Problem definition:** Decide the task (regression/classification/clustering) and success metric. *E.g., "Predict house price within ±5%".*
2. **Data collection:** CSV/Excel files, databases, APIs, web scraping, sensors, surveys.
3. **Data cleaning/preprocessing:** Fix missing values, outliers, duplicates, wrong types, inconsistent formats ("Delhi" vs "delhi").
4. **EDA:** Study distributions, relationships, correlations.
5. **Feature engineering:** Create/transform/select features (e.g., age from date of birth, one-hot encoding, scaling).
6. **Model selection:** Choose algorithm(s) suited to the problem and data size.
7. **Training:** Fit model on the **training set**.
8. **Evaluation:** Measure on the **test set** with appropriate metrics.
9. **Hyperparameter tuning:** Grid search, cross-validation.
10. **Deployment & monitoring:** Serve via an app/API, watch for *data drift*.

Data split: Train (≈70–80%) learns · Validation (≈10–15%) tunes · Test (≈10–20%) final unbiased check.

1.7 Challenges in ML

Insufficient/poor-quality data · Non-representative data (sampling bias) · Irrelevant features · **Overfitting** (memorizes training data) · **Underfitting** (too simple) · Bias & fairness · Data privacy.

---

2. Types of Machine Learning

2.1 Supervised Learning

- Trained on **labeled** data (X, y). The "teacher" gives correct answers.
- **Regression:** continuous output (price, temperature). *Algos:* Linear Regression, Ridge, SVR.
- **Classification:** categorical output (spam/ham). *Algos:* Logistic Regression, k-NN, Decision Tree,

SVM, Naive Bayes.

- *Examples:* house price prediction, email spam detection, disease diagnosis, loan default prediction.

2.2 Unsupervised Learning

- Trained on **unlabeled** data (only X); finds hidden structure.
- **Clustering:** group similar items (k-Means, Hierarchical, DBSCAN).
- **Dimensionality reduction:** compress features (PCA).
- **Association:** find item relations (Apriori: "bread → butter").
- *Examples:* customer segmentation, anomaly detection, topic discovery.

2.3 Semi-Supervised Learning

- Uses a **small labeled set + large unlabeled set**. Useful when labeling is expensive (needs experts).
- *Techniques:* self-training (model labels unlabeled data with confident predictions), label propagation.
- *Examples:* medical images (few labeled by doctors), speech recognition, web page classification.

2.4 Reinforcement Learning (for awareness)

An **agent** interacts with an **environment**, takes **actions**, gets **rewards/penalties**, and learns a policy to maximize total reward. *Examples:* AlphaGo, robot walking, self-driving.

2.5 Comparison

	Supervised	Unsupervised	Semi-supervised
Labels	All	None	Few
Goal	Predict	Discover patterns	Predict with less labeling cost
Feedback	Direct	None	Partial
Example	Price prediction	Clustering	Image labeling
Cost of labeling	High	None	Moderate

---

3. Understanding Data

3.1 Structured vs Unstructured

	Structured	Unstructured	Semi-structured
Format	Rows & columns, fixed schema	No predefined format	Tags/keys, flexible
Storage	SQL, CSV, Excel	Files, data lakes, NoSQL	JSON, XML
Examples	Bank transactions, student marks	Images, audio, video, emails, tweets	API responses, HTML

	marks	tweets	HTML
Analysis	Easy (SQL, Pandas)	Needs special ML (CNN, NLP)	Parsing needed
Share of world data	~20%	~80%	

3.2 Types of Features

```
Features
├── Numerical
│   ├── Discrete (counts: no. of children = 0,1,2)
│   └── Continuous (height = 165.4 cm)
└── Categorical
    ├── Nominal (no order: colour, city)
    └── Ordinal (order: low < medium < high)
```

Also: **Binary** (yes/no), **Datetime**, **Text**.

Data scales (Stevens): Nominal → Ordinal → Interval (no true zero, e.g., °C) → Ratio (true zero, e.g., weight).

3.3 Common Data Sources

CSV/Excel · SQL databases · APIs/JSON · Web scraping (BeautifulSoup) · Kaggle/UCI repository · built-in sklearn datasets (`load_iris` , `load_diabetes`).

---

4. Data Loading & Basic Operations with Pandas

Pandas provides two structures: **Series** (1-D labeled array) and **DataFrame** (2-D table).

4.1 Creating Data

```
import pandas as pd
import numpy as np

s = pd.Series([10, 20, 30], index=["a", "b", "c"])      # Series

data = {"Name": ["Amit", "Riya", "Sam", "Neha"],
        "Age":  [21, 22, np.nan, 23],
        "Marks":[85, 90, 78, 92],
        "City": ["Delhi", "Pune", "Delhi", "Goa"]}
df = pd.DataFrame(data)                                  # DataFrame
```

4.2 Loading & Saving

```
df = pd.read_csv("students.csv")                 # CSV
df = pd.read_csv("file.csv", sep=";", header=0, usecols=["Name","Marks"], nrows=100)
df = pd.read_excel("file.xlsx", sheet_name="Sheet1")
df = pd.read_json("file.json")
df = pd.read_sql("SELECT * FROM t", connection)

df.to_csv("out.csv", index=False)
```

```
df.to_excel("out.xlsx", index=False)
```

4.3 Inspecting Data

```
df.head(); df.tail(3)          # first / last rows
df.shape                       # (rows, columns)
df.size, df.ndim
df.columns, df.index
df.dtypes
df.info()                      # types + non-null counts + memory
df.describe()                  # count, mean, std, min, 25%, 50%, 75%, max
df.describe(include="object")  # for categorical columns
df.nunique(); df["City"].unique(); df["City"].value_counts()
df.sample(5)                   # random rows
```

4.4 Selecting & Filtering

```
df["Marks"]                    # Series
df[["Name", "Marks"]]          # DataFrame
df.loc[0]                      # row by label
df.loc[0:2, ["Name","Age"]]    # label-based (end INCLUSIVE)
df.iloc[0:2, 0:2]              # position-based (end EXCLUSIVE)

df[df["Marks"] > 80]
df[(df["Marks"] > 80) & (df["City"] == "Delhi")]     # AND
df[(df["City"] == "Goa") | (df["City"] == "Pune")]   # OR
df[df["City"].isin(["Delhi", "Goa"])]
df.query("Marks > 80 and City == 'Delhi'")
```

4.5 Modifying Data

```
df["Grade"] = np.where(df["Marks"] >= 90, "A", "B")      # new column
df["Marks"] = df["Marks"] + 5
df.rename(columns={"Marks": "Score"}, inplace=True)
df.drop(columns=["Grade"], inplace=True)                 # drop column
df.drop(index=[0, 1], inplace=True)                      # drop rows
df["Age"] = df["Age"].astype("float")                    # change dtype
df["City"] = df["City"].str.lower().str.strip()          # string ops
df["Marks"].apply(lambda x: x * 2)                       # apply function
df.reset_index(drop=True, inplace=True)
```

4.6 Sorting, Grouping, Aggregation, Merging

```
df.sort_values("Marks", ascending=False)
df.sort_values(["City", "Marks"], ascending=[True, False])

df.groupby("City")["Marks"].mean()
df.groupby("City").agg({"Marks": ["mean", "max"], "Age": "min"})
pd.crosstab(df["City"], df["Grade"])
df.pivot_table(values="Marks", index="City", aggfunc="mean")

# Combining DataFrames
pd.concat([df1, df2], axis=0)                        # stack rows
pd.merge(df1, df2, on="ID", how="inner")             # SQL-like join (inner/left/right/oute
```

---

5. Data Cleaning

> "Garbage in, garbage out." Data scientists spend ≈60–80% of time cleaning data.

5.1 Handling Missing Values

Why missing? Human error, sensor failure, non-response in surveys, merging errors.

Types:

- **MCAR** (Missing Completely At Random): no pattern. Safe to drop.
- **MAR** (Missing At Random): depends on other observed columns (e.g., older people skip the income field).
- **MNAR** (Missing Not At Random): depends on the missing value itself (high earners hide income).

Detect:

```
df.isnull().sum()                          # count per column
(df.isnull().mean() * 100).round(2)        # % missing
sns.heatmap(df.isnull(), cbar=False)       # visual map
```

Strategies:

Method	When to use	Code
Drop rows	Few rows missing (<5%), MCAR	`df.dropna()`
Drop columns	>40–50% missing	`df.drop(columns=["col"])`
Mean imputation	Numeric, symmetric	`df["Age"].fillna(df["Age"].mean())`
Median imputation	Numeric, skewed/outliers	`df["Age"].fillna(df["Age"].median())`
Mode imputation	Categorical	`df["City"].fillna(df["City"].mode()[0])`
Constant	Domain-specific	`fillna("Unknown")`
Forward/backward fill	Time series	`df.ffill()`, `df.bfill()`
Interpolation	Ordered numeric data	`df["x"].interpolate()`
KNN imputer	Use similar rows	`KNNImputer(n_neighbors=3)`

```
df.dropna()                            # any NaN in row
df.dropna(how="all")                   # only fully empty rows
df.dropna(subset=["Age"])              # NaN in specific column
df.dropna(axis=1, thresh=int(0.6*len(df)))   # keep columns with >=60% data

from sklearn.impute import SimpleImputer, KNNImputer
```

```
imp = SimpleImputer(strategy="median")        # mean / median / most_frequent / constant
df[["Age"]] = imp.fit_transform(df[["Age"]])

knn_imp = KNNImputer(n_neighbors=3)
df_num = pd.DataFrame(knn_imp.fit_transform(df.select_dtypes("number")),
                      columns=df.select_dtypes("number").columns)
```

⚠ **Data leakage:** fit the imputer on the **training** set only, then transform both train and test.

5.2 Handling Outliers

Outlier: a value far from the rest. *Causes:* data entry error (age = 250), measurement error, genuine extreme (a billionaire's income). **Effects:** distort mean, std, correlation and models like linear regression; k-NN is also affected.

Detection Methods

(a) Visual: box plot, scatter plot, histogram.

```
sns.boxplot(x=df["Salary"])
```

(b) Z-score: z = (x − μ) / σ → outlier if |z| > 3 (works if data is roughly normal).

```
from scipy import stats
z = np.abs(stats.zscore(df["Salary"]))
outliers = df[z > 3]
```

(c) IQR method (robust):

- IQR = Q3 − Q1
- Lower fence = Q1 − 1.5×IQR; Upper fence = Q3 + 1.5×IQR

```
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]
```

Worked example: data = 2, 4, 5, 6, 7, 8, 40. Q1 = 4, Q3 = 8, IQR = 4 → fences: −2 and 14 → **40 is an outlier**.

(d) Advanced: Isolation Forest, DBSCAN, LOF.

Treatment

```
# 1. Remove
df_clean = df[(df["Salary"] >= lower) & (df["Salary"] <= upper)]

# 2. Cap / Winsorize
df["Salary"] = df["Salary"].clip(lower, upper)

# 3. Transform (reduces skew)
df["Salary_log"] = np.log1p(df["Salary"])

# 4. Replace with median
```

```
# 4. Replace with median
df.loc[df["Salary"] > upper, "Salary"] = df["Salary"].median()
```

Decision: if error → fix/remove; if genuine and important (fraud!) → keep or use robust models.

5.3 Handling Duplicate Values

Duplicates bias the model and inflate accuracy (same row in train and test).

```
df.duplicated().sum()                          # count full-row duplicates
df[df.duplicated(keep=False)]                  # view all duplicates
df.drop_duplicates(inplace=True)               # keep first
df.drop_duplicates(subset=["Email"], keep="last", inplace=True)
```

5.4 Other Cleaning Tasks

```
df["Date"] = pd.to_datetime(df["Date"])                    # fix datatypes
df["Price"] = pd.to_numeric(df["Price"], errors="coerce")  # "abc" → NaN
df["City"] = df["City"].str.strip().str.title()            # inconsistent text
df["Gender"] = df["Gender"].replace({"M": "Male", "F": "Female"})
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

Cleaning checklist: ① dtypes ② missing ③ duplicates ④ outliers ⑤ inconsistent labels ⑥ invalid values (negative age).

---

6. Exploratory Data Analysis (EDA): Statistical Techniques

EDA (John Tukey) = investigating a dataset to summarize its main characteristics, detect anomalies, test assumptions, and find patterns *before* modelling.

6.1 Steps of EDA

1. Understand the dataset (shape, dtypes, meaning of columns)
2. Clean data
3. **Univariate** analysis (one variable)
4. **Bivariate** analysis (two variables)
5. **Multivariate** analysis (≥3 variables)
6. Document insights/hypotheses

6.2 Measures of Central Tendency

Measure	Formula	Use
Mean	Σx/n	Symmetric data
Median	middle value (sorted)	Skewed data, outliers
Mode	most frequent	Categorical data

Example: data 3, 5, 7, 9, 100 → mean = 24.8 (pulled up by 100), median = 7 (robust).

6.3 Measures of Dispersion (Spread)

Measure	Formula
Range	max – min
Variance	$\sigma^2 = \Sigma(x - \mu)^2 / n$ (sample: n – 1)
Std deviation	$\sigma = \sqrt{\text{variance}}$
IQR	Q3 – Q1
Coefficient of Variation	$\sigma / \mu \times 100\%$

Quartiles: Q1 (25th percentile), Q2 (median), Q3 (75th). **Five-number summary:** min, Q1, median, Q3, max (basis of the box plot).

6.4 Shape of Distribution

- **Skewness:** asymmetry. *Positive (right) skew:* mean > median > mode (income). *Negative (left):* mean < median. *≈0:* symmetric.
- **Kurtosis:** tail heaviness/peakedness. Normal distribution kurtosis (excess) = 0.
- **Normal distribution:** bell curve; 68–95–99.7 rule (±1σ, ±2σ, ±3σ).

6.5 Relationship Measures

Covariance: $\text{Cov}(X, Y) = \Sigma(x - \bar{x})(y - \bar{y}) / (n-1)$. Sign shows direction; magnitude depends on units.

Pearson correlation coefficient: $r = \text{Cov}(X, Y) / (\sigma_x \sigma_y)$, $-1 \le r \le +1$

r	Interpretation
+0.7 to +1	strong positive
+0.3 to +0.7	moderate positive
≈ 0	no linear relation
–1 to –0.3	negative

- **Spearman** correlation: rank-based, for monotonic/non-linear/ordinal data.
- ⚠ **Correlation ≠ causation** (ice-cream sales and drowning both rise in summer).
- **Multicollinearity:** features highly correlated with each other ($|r| > 0.8$) harm linear models.

6.6 Categorical Analysis

Frequency tables, `value_counts(normalize=True)`, cross-tabulation, **Chi-square test** for association between two categorical variables.

6.7 EDA in Code

```
import pandas as pd, numpy as np
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True); df = iris.frame

# Overview
```

```
df.shape; df.info(); df.describe()

# Univariate
df["sepal length (cm)"].agg(["mean", "median", "std", "var", "min", "max"])
df["sepal length (cm)"].skew(); df["sepal length (cm)"].kurt()
df["target"].value_counts(normalize=True)

# Bivariate
df.corr()                                          # Pearson matrix
df.corr(method="spearman")
df.groupby("target").mean()                        # compare groups

# Multivariate
pd.pivot_table(df, values="petal length (cm)", index="target", aggfunc=["mean","std"])

# Hypothesis tests (scipy)
from scipy import stats
stats.pearsonr(df["petal length (cm)"], df["petal width (cm)"])   # (r, p-value)
stats.ttest_ind(group1, group2)                                    # compare two means
pd.crosstab(df["A"], df["B"]).pipe(lambda t: stats.chi2_contingency(t))
```

p-value: probability of seeing the result if the null hypothesis is true. If p < 0.05 → statistically significant.

---

7. Data Visualization: Matplotlib & Seaborn

Why? Humans grasp patterns visually; plots reveal outliers, trends, and relationships that numbers hide (e.g., *Anscombe's quartet*).

7.1 Matplotlib (foundation library)

Anatomy: **Figure** (canvas) → **Axes** (single plot) → Axis, title, labels, legend, ticks.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(1, 6); y = x ** 2

plt.figure(figsize=(6, 4))
plt.plot(x, y, color="green", linestyle="--", marker="o", label="y = x²")
plt.title("Line Plot"); plt.xlabel("x"); plt.ylabel("y")
plt.legend(); plt.grid(True)
plt.savefig("plot.png", dpi=200)
plt.show()
```

Common plots:

```
plt.bar(["A","B","C"], [10, 25, 15])               # bar chart
plt.barh(["A","B","C"], [10, 25, 15])              # horizontal bar
plt.hist(data, bins=20, edgecolor="black")         # histogram
plt.scatter(x, y, c="red", s=50, alpha=0.6)        # scatter
plt.pie([40, 30, 30], labels=["X","Y","Z"], autopct="%1.1f%%")   # pie
plt.boxplot([data1, data2], labels=["G1","G2"])    # box plot

# Subplots
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].hist(data);  axes[0].set_title("Histogram")
axes[1].boxplot(data); axes[1].set_title("Box")
axes[2].scatter(x, y); axes[2].set_title("Scatter")
plt.tight_layout(); plt.show()
```

7.2 Seaborn (statistical visualization)

Built on Matplotlib; works directly with DataFrames; attractive defaults.

```
import seaborn as sns
sns.set_theme(style="whitegrid")
tips = sns.load_dataset("tips")        # built-in dataset

# Distribution plots (univariate numeric)
sns.histplot(tips["total_bill"], kde=True, bins=20)
sns.kdeplot(tips["total_bill"], fill=True)

# Categorical plots
sns.countplot(x="day", data=tips)                      # frequency
sns.barplot(x="day", y="total_bill", data=tips)        # mean ± CI
sns.boxplot(x="day", y="total_bill", data=tips)        # median, quartiles, outliers
sns.violinplot(x="day", y="total_bill", data=tips)     # distribution shape

# Relational plots (bivariate)
sns.scatterplot(x="total_bill", y="tip", hue="sex", size="size", data=tips)
sns.lineplot(x="size", y="tip", data=tips)
sns.regplot(x="total_bill", y="tip", data=tips)        # scatter + regression line

# Multivariate
sns.pairplot(tips, hue="sex")                          # all pairs
sns.heatmap(tips.corr(numeric_only=True), annot=True, cmap="coolwarm", vmin=-1, vmax=1)
```

7.3 Reading a Box Plot

```
 Outliers  ○     |-----[  Q1 | median | Q3  ]-----|     ○
               lower whisker      box (IQR)     upper whisker
```

Whiskers extend to the furthest points within 1.5×IQR; points beyond are outliers.

7.4 Choosing the Right Plot

Goal	Plot
Distribution of 1 numeric variable	Histogram, KDE, box plot
Frequency of categories	Count/bar plot, pie
Compare numeric across categories	Box, violin, bar
Relationship between 2 numeric	Scatter, regplot
Trend over time	Line plot
Correlation of many variables	Heatmap, pairplot

Detect outliers	Box plot, scatter
Missing data	Heatmap of `isnull()`

7.5 Good Visualization Practice

Always label axes + title + units · choose suitable chart type · avoid clutter and 3-D effects · use colour meaningfully (colour-blind safe palettes) · start bar-chart axes at zero.

7.6 Mini End-to-End Example

```
import pandas as pd, numpy as np, seaborn as sns, matplotlib.pyplot as plt

df = sns.load_dataset("titanic")

# 1. inspect
print(df.shape); print(df.isnull().sum())

# 2. clean
df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df = df.drop(columns=["deck"]).drop_duplicates()

# 3. EDA
print(df.describe())
print(df.groupby("sex")["survived"].mean())

# 4. visualize
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
sns.countplot(x="survived", hue="sex", data=df, ax=ax[0])
sns.histplot(df["age"], kde=True, ax=ax[1])
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm", ax=ax[2])
plt.tight_layout(); plt.show()
```

---

8. Unit 1 Revision & Practice

Quick Revision

- ML = learning patterns from data; workflow: collect → clean → EDA → features → train → evaluate → tune → deploy.
- Supervised (labeled) · Unsupervised (unlabeled) · Semi-supervised (few labels).
- Pandas: `read_csv`, `info`, `describe`, `loc/iloc`, `groupby`.
- Missing → drop/impute · Outliers → IQR/Z-score, cap/remove · Duplicates → `drop_duplicates`.
- EDA: mean/median/mode, variance, skew, correlation.
- Plots: histogram (distribution), box (outliers), scatter (relation), heatmap (correlation).

Short Questions

1. Define ML. Differentiate it from traditional programming.
2. What are features and labels?
3. Difference between structured and unstructured data (with examples).

4. Why is the median preferred over the mean for skewed data?
5. What is data leakage?
6. State the IQR rule for outlier detection.
7. Difference between `loc` and `iloc`.
8. What does correlation of −0.9 mean?

Long Questions

1. Explain the ML workflow with a diagram.
2. Explain types of ML with examples and a comparison table.
3. Describe techniques for handling missing values, outliers and duplicates with Python code.
4. Explain EDA and its statistical techniques.
5. Discuss Matplotlib and Seaborn with plots and when to use each.

Numerical Practice

Data: 12, 15, 14, 10, 8, 12, 14, 13, 90. Find the mean, median, mode, and test for outliers by IQR.
Answer outline: sorted = 8, 10, 12, 12, 13, 14, 14, 15, 90 → mean ≈ 20.9, median = 13, mode = 12 and 14. Q1 = 11, Q3 = 14.5, IQR = 3.5, upper fence = 19.75 → **90 is an outlier**.